BAWABU

Bawabu, explained simply

What we have built so far (Steps 0 to 3), how the pieces fit together, and what Modal and Baseten are for.

The whole thing in 30 seconds

Your AI coding helper (Muse Code) installs software packages all by itself. A few packages on the internet are booby-trapped: the moment they install, they steal your passwords and keys.

Bawabu (Swahili for *gatekeeper*) stands at your door. Every time the helper tries to bring a package in, Bawabu stops it, asks a security office in the cloud whether it is safe, and only then lets it in. If anything goes wrong, Bawabu keeps the door shut.

What works today	
The guard	Stops every install a real Muse agent tries, including ones hidden in scripts and new MCP servers
The office	Runs in the cloud on Modal and makes real decisions in about a second: blocks made-up and known-malicious packages, allows established ones
The notebook	Remembers every answer in Supabase, shared across machines; answers expire so later malware reports are noticed
The detective	Tested and ready to plug in (Meta's Muse Spark and Baseten's Spark Open)
The lab	Tested and ready to plug in (Modal sandboxes)

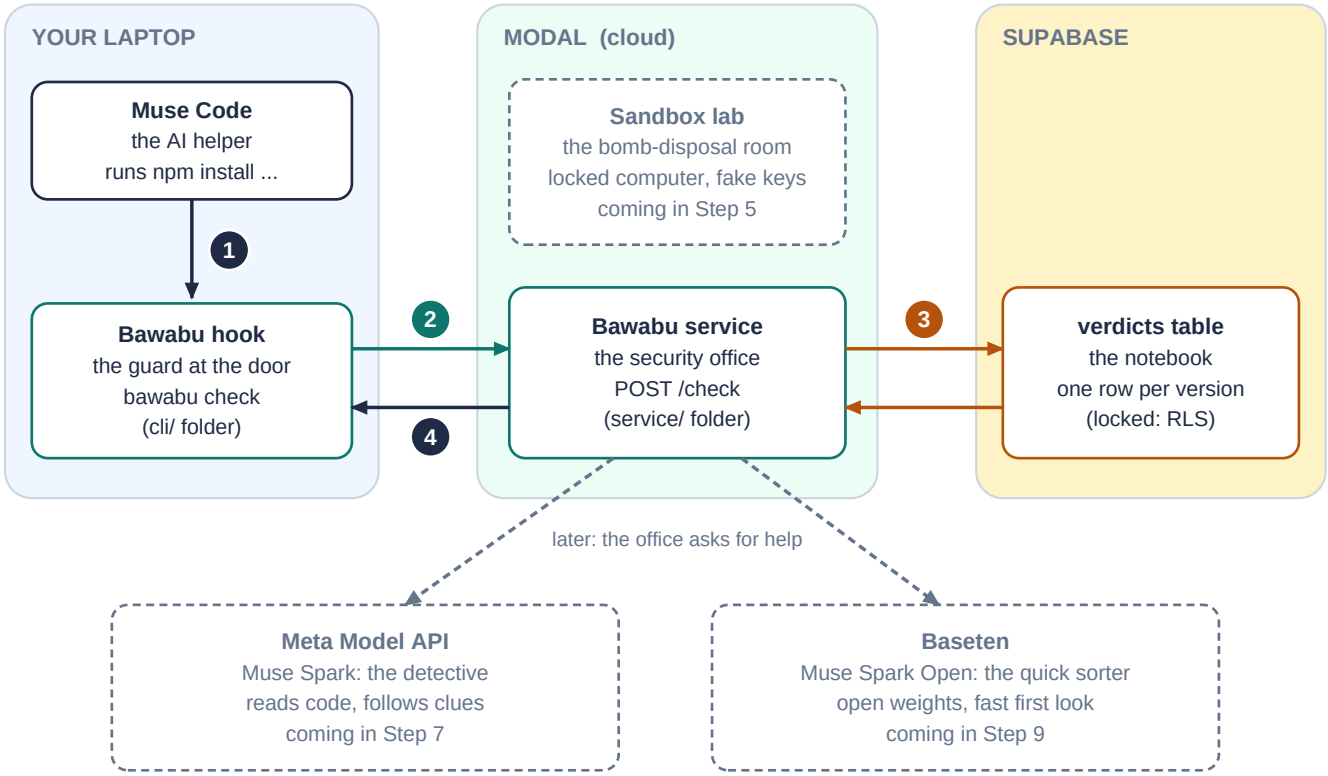
Meta Global AI Developer Hackathon • Indie track • 29 October 2026

1. The story: parcels at your front door

Imagine your computer is your house. Your AI helper keeps ordering parcels (software packages) from strangers on the internet and opening them inside. Almost all parcels are fine. A few are traps. Here is who is who:

In the story	In real life	What it does
The helper who orders parcels	Muse Code, Meta's AI coding agent	Writes code and runs commands like `npm install left-pad` on its own.
The parcels	Packages from npm (JavaScript) and PyPI (Python)	Ready-made code that other people publish. Most are fine; a few are traps.
The guard at the front door	The Bawabu hook (`bawabu check`)	Runs on your laptop before every command. Sees every parcel the helper orders and can say **stop**.
The security office	The Bawabu service, running on Modal	A small program in the cloud. The guard phones it: *is left-pad version 1.3.0 safe?* It answers **allow**, **block** or **ask**.
The office notebook	Supabase, a database	Remembers every answer, so the same parcel is answered instantly next time, for everyone.
The bomb-disposal room	Modal Sandboxes (coming, Step 5)	Opens a suspicious parcel in a sealed room with fake jewellery inside, and films what it tries to steal.
The detective	Muse Spark, via the Meta Model API (coming, Step 7)	Reads the parcel's insides, follows clues and writes the verdict with evidence.
The quick sorter	Muse Spark Open, on Baseten (coming, Step 9)	Glances at parcels first. Normal ones go straight through; odd ones go to the detective.

2. The big picture



1	Before every command, Muse hands it to the guard.
2	The guard sends only the package name and version: *is left-pad 1.3.0 safe?*
3	The office looks it up in the notebook, and saves any new answer.
4	The answer comes back: **allow**, **block** or **ask**.

Solid boxes and arrows work today. Dashed boxes are tested on their own and get connected in later steps.

3. What happens when the helper types `npm install left-pad`

1	**Muse wants to run a command** The helper decides to run `npm install left-pad`. Before running anything, Muse calls our guard (a *PreToolUse hook*) and hands it the command.
2	**The guard reads the command** It reads the command the way a shell would and finds the parcel: **left-pad** from npm. It also looks inside anything the command would run: script files like setup.sh, package.json scripts, and MCP servers being added to settings.
3	**Ordinary commands go straight through** Commands that install nothing (`git status`, `ls`) are waved through instantly. So are packages on your allowlist and your own local code.
4	**The guard phones the office** It sends **only the package name and version** to the service. Never the command itself, because commands can contain passwords.
5	**The office checks its notebook** If this exact version was judged before, the answer comes back at once. If not, the office runs its **quick checks** (section 8) and writes the answer down.
6	**The answer comes back** **Allow:** the command runs. **Block** or **ask:** the guard stops it and tells the helper why, and that it must not try another way.
7	**Everyone knows what happened** The helper tells you. Every decision is also written to a log on your laptop: `~/.local/state/bawabu/checks.jsonl`.

Measured: the whole round trip, from the guard to the cloud office and back, takes **about 2 seconds**. The first call after the office has been idle takes about 5 seconds, because it has to wake up (a *cold start*).

4. The safety rules we built in

If in doubt, keep the door shut.	We discovered that Muse quietly lets a command run if the guard is too slow, crashes, or answers unclearly. So Bawabu says **stop** whenever anything goes wrong: the office can't be reached, the check takes too long, the settings are broken, or the command does something Bawabu can't see into (like downloading a script and piping it straight into the shell).
The helper can't fire the guard.	Bawabu's settings live in the project's `.muse` folder, which the helper can't change without you approving each edit. A tricked helper can't switch Bawabu off or put its own malware on the allowlist.
Only names and versions leave your laptop.	The office never sees your commands, your code or your files.
Only the office can write in the notebook.	The database table is locked (Supabase *row-level security*). We tested it: a forged "allow" written from outside was refused.
Old answers aren't trusted forever.	Each answer records which version of the checks made it (the *engine*) and when it expires. The Step 2 placeholder answers (engine 0) are now ignored, and answers about young versions are re-checked within 6 hours, so a malware report that arrives later is noticed.
Clear time limits.	Muse waits up to 180 s for the guard (we tested that it really does). The guard gives up at 150 s and blocks; it waits at most 140 s for the office.

5. What Modal does

Modal is a cloud computer you rent by the second. You give it Python code, and it runs that code on its own machines. When nobody is using it, it sleeps and costs nothing. Bawabu uses Modal for three jobs:

Job	In the story	Details
1. Runs the security office	The office building	The Bawabu service lives at `https://amohntim123--bawabu.us-east.modal.run`. It wakes up on the first call (about 5 s), then answers in under a second. The recipe is `service/modal_app.py`; we publish it with `uv run modal deploy service/modal_app.py`.
2. Keeps the office's keys	The locked key cabinet	The office needs the Supabase key to use the notebook, but it can't read your `.env` file, which only exists on your laptop. So the key goes in a **Modal secret** (`bawabu-supabase`), which Modal hands to the service when it starts. The Meta and Baseten keys will get secrets of their own.
3. Runs the bomb-disposal room (Step 5)	The sealed room with cameras	**Modal Sandboxes** are throwaway locked computers. Bawabu will open suspicious packages there, with fake passwords planted as bait. Tested on 28 Oct: Node runs, the internet is fully blocked (even looking up names), and *strace* recorded the test script starting a shell and reading the fake token. Ready in about 4 to 6 s.

6. What Baseten does

Baseten runs AI models for you, so you don't need your own graphics cards. The hackathon gave us access to **Muse Spark Open** (`meta-models/muse-spark-open-1026`): the *open-weights* version of Meta's Muse Spark, meaning the model's recipe is public and anyone could run it themselves.

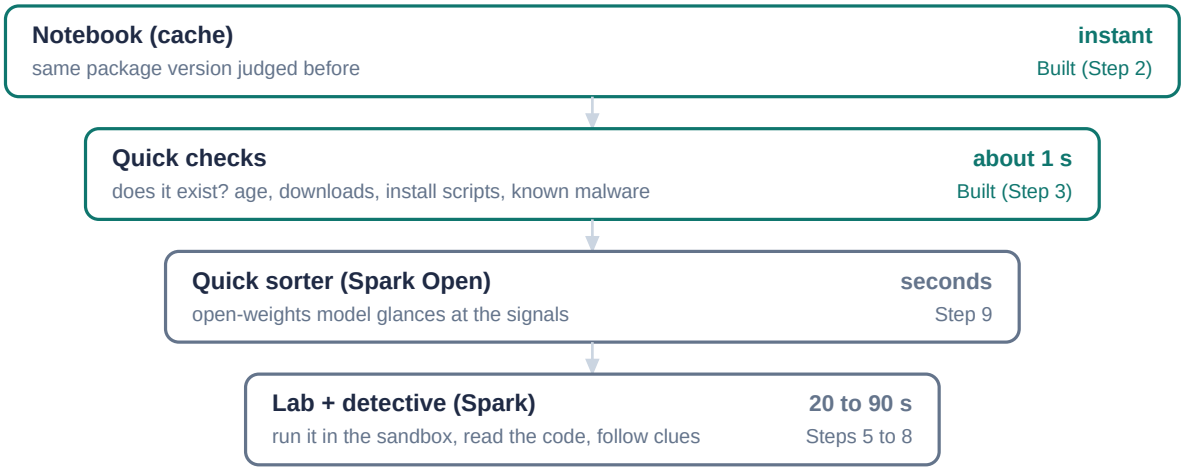
Job	Details
1. The quick sorter (Step 9)	The second layer of the funnel (section 7). It reads a short summary of a new package and decides *looks normal* or *send to the detective*. Speed matters here, and in our test it was about twice as fast as Meta's API (chat 1.5 s, a tool call 4.2 s).
2. The open-weights mode	A `--open-weights` option that runs the whole detective on the open model, for people who prefer open models. It also targets the hackathon's **Best Build on Meta Open Weight Models** prize ($10K).
Tested 27 Oct	Passed all four checks: the model is listed, plain chat, calling a tool and using the result, and answering in a fixed format. Your Baseten key only works with that exact model name.

And Meta's Model API: the detective

Your $150 of Meta credits pay for **Muse Spark**, the main investigator (Step 7). On 27 Oct we proved it can do what a detective needs: ask to use a tool (we gave it a *decode* tool and a hidden malicious payload), read the result, and give a verdict in a fixed format (it said *block*, because "a date-picker has no legitimate reason" to read your npm token). It is slow mostly because it thinks before answering: on the *minimal* thinking setting it took 2.3 s instead of 7 s, with the same verdict.

7. The funnel: cheap checks first

Checking every parcel with the detective would be slow and expensive. So parcels go through layers, cheapest first, and most stop at the top. Only the strange ones reach the lab and the detective.



Each layer only sees the parcels the layer above couldn't settle.

8. How the office decides: the quick checks (Step 3)

Before any slow, expensive testing, the office reads the parcel's paperwork, like a customs officer checking a passport. It takes about a second and never opens the parcel. It asks seven questions:

Question	Where the answer comes from	In the story
Is it known malware?	OSV.dev, a public list of malicious packages	Is this parcel on the police's wanted list? Checked even if the shop has already pulled it.
Does it exist at all?	The npm or PyPI registry	Is there really a shop with this name? If not, the helper made the name up.
Which exact version?	The registry, using npm's and pip's own rules	Not just "the latest parcel" but exactly which one, so the notebook entry is precise.
How old and how popular?	The registry and weekly download counts	How long has the shop been open, and how many customers does it have?
Does it run code when installed?	The package's install scripts	Does the parcel open itself? And did it only start doing that in this version?
Same sender as before?	Provenance (a CI signature) and who published it	Is the seal on the box the same as last time, and is it the usual courier?
Is the name a near-copy of a famous one?	Lists of the 17,000 most popular npm names and 15,000 most popular PyPI names	Is "Amazn" pretending to be Amazon?

What the office answers

What it finds	Answer	Notebook keeps it for
Known malware, or npm has taken the package down	**block**	30 days
The package doesn't exist (with a "did you mean" when it can tell)	**block**	Not kept: someone could register the name later
The version asked for doesn't exist	**block**	Not kept
Any red flag: brand new, very few downloads, install code that is new or changed, a lost signature, a new publisher, a near-copy name	**ask** (for now)	6 hours
Nothing unusual	**allow**	7 days (6 hours if the version is under a week old)

For now, **ask** means the helper must check with you. From Step 5, red-flag packages go to the lab and the detective instead. Answers expire so that if a package is reported as malware later, the office notices.

"Did you mean...?"

When the helper asks for a package that doesn't exist, the office first fixes spelling mistakes using the words found in popular package names ("pickr" becomes "picker"), then finds the closest popular package. This takes about 40 milliseconds.

The helper asked for	The office suggests
`react-datepickr`	`react-datepicker`
`prisma-clinet`	`@prisma/client`

The helper asked for	The office suggests
`axois`	`axios`
`mongose`	`mongoose`
`expresss-validator`	`express-validator`
`fancy-date-pickr`	Nothing. Changing "fancy" to "react" is a guess about meaning, not spelling: a job for the detective (Step 7).

Tested on real packages (29 October)

Package	Answer	Why
`react`	allow	15 years old, 205 million downloads a week, no install code. About 1 second.
`esbuild`	allow	It runs install code, but the same code it has always run.
`react-datepickr`	block	Doesn't exist. "Did you mean react-datepicker?"
`@ctrl/tinycolor@4.1.1`	block	Planted by the Shai-Hulud worm in 2025. npm deleted it, but OSV still lists it as malware.
`crossenv`	block	A 2017 typosquat that npm replaced with a security placeholder.
`requests` (PyPI)	allow	15 years old, 330 million downloads a week.

All 14 test packages got the expected answer. One gap remains: installing a whole project at once (`npm install` with no package names) isn't checked yet, because the office can't see the project's list of dependencies. That's next on the list.

9. The files, one by one

Everything lives in your **bawabu** folder (and on GitHub). Here is what each file is for.

The top of the folder

File	What it is
`plan.md`	The build plan you wrote: the problem, the design, the schedule.
`results.md`	Our lab notebook: every test, result and decision, newest first. Updated as we go.
`.env`	Your secret keys (Meta, Baseten, Supabase). **Only on your laptop**; never uploaded.
`.env.example`	An empty copy of `.env` that is safe to upload, so others know which keys are needed.
`.gitignore`	Tells git what must never be uploaded: `.env`, downloaded libraries, build output.
`pyproject.toml`, `uv.lock`, `.python-version`	The Python project: which libraries we use (FastAPI, Modal, the OpenAI client, httpx) and Python 3.13. The *uv* tool reads these.

`cli/`: the guard (TypeScript, runs on the developer's laptop)

File	What it does
`src/cli.ts`	The front door of the command: `bawabu init`, `bawabu check`, `bawabu explain`.
`src/init.ts`	`bawabu init`: installs the guard in a project, writes the settings, then tests the guard exactly the way Muse runs it, and checks the office is reachable.
`src/check.ts`	**The guard itself.** What Muse runs before every command: decides allow or block, phones the office, and blocks on any failure.
`src/shell.ts`	Reads a command line like a shell does: quotes, `&&`, pipes, heredocs, `$( )`.
`src/managers.ts`	Knows the package managers (npm, npx, pnpm, yarn, bun, pip, uv, pipx) and which packages each command would fetch.
`src/parse.ts`	Follows a command into the code it runs (script files, package.json scripts) and flags anything it can't see into.
`src/mcp.ts`	Spots new MCP servers being added to a settings file, and what they would download.
`src/service.ts`	Talks to the office: sends package names and versions, reads the verdict.
`src/config.ts`	Reads the project's settings (`.muse/bawabu.json`: mode, office address, allowlist) and checks they're valid.
`src/constants.ts`	Fixed numbers: the 180, 150 and 140 second limits, and the office's default address.
`src/*.test.ts`	50 automatic tests (`npm test`). All passing.
`package.json`, `tsconfig.json`, `.npmrc`	Project setup. `.npmrc` stops our own libraries from running install scripts, because we practise what we preach.

`service/`: the security office (Python, runs on Modal)

File	What it does
`modal_app.py`	Tells Modal how to run the office: which software to install, which secret to use, and the web address.
`bawabu_service/api.py`	The office's two doors: `GET /health` (are you alive?) and `POST /check` (is this package safe?).
`bawabu_service/models.py`	The exact shape of the questions and answers, shared with the guard.
`bawabu_service/verdicts.py`	How a package gets its verdict: the notebook first, then the quick checks. Holds the engine number (now 1).
`bawabu_service/quick_checks.py`	**The quick checks' rules:** turns the facts into allow, block or ask, with a reason and any red flags.
`bawabu_service/registry.py`	Asks npm and PyPI about a package: does it exist, which exact version, its age, install code, signature and publisher.
`bawabu_service/osv.py`	Asks OSV.dev whether the package is known malware.
`bawabu_service/popular.py`	Popular names: spots near-copy names and works out "did you mean...?".
`bawabu_service/popular_npm.py`	The list of 17,338 popular npm names (from npm-high-impact, MIT licence).
`bawabu_service/net.py`	The office's internet settings: time limits and how it identifies itself to registries.
`bawabu_service/cache.py`	The notebook: the real one in Supabase, or a pretend one in memory for tests. Answers carry their red flags and an expiry time.
`tests/`	60 automatic tests (`uv run pytest`), run against a fake internet so they're fast and repeatable. All passing.
`scripts/smoke_checks.py`	The Step 3 test against the real npm, PyPI and OSV (14 packages).
`scripts/update_popular.py`	Refreshes the popular npm names list.
`scripts/smoke_spark.py`	The Step 0 test that proved Meta's API and Baseten work (chat, tools, fixed-format answers).

`sandbox/`: the lab

File	What it does
`scripts/smoke_modal.py`	The Step 0 test that proved Modal's locked room works: Node runs, no internet, strace sees everything.

Outside the folder

Where	What it is
Supabase: `public.verdicts`	The notebook. One row per package version: the answer, the reason, any red flags, a suggested package, the engine that made it, and when it expires. Locked so only the office can write.
Each project, after `bawabu init`	`.muse/hooks.json` tells Muse to call the guard before every tool call. `.muse/bawabu.json` holds Bawabu's settings.
`~/.local/state/bawabu/checks.jsonl`	A log on your laptop of every decision the guard made.

10. Where the keys live

Key	On your laptop	In the cloud	Used for
Supabase secret key	`.env`	Modal secret `bawabu-supabase`	The office reading and writing the notebook (now)
Meta API key	`.env`	A Modal secret, later	The detective, Muse Spark (Step 7)
Baseten API key	`.env`	A Modal secret, later	The quick sorter, Spark Open (Step 9)

Keys never go in the code (the repository is public) and never in the database.

11. Progress

Step	Status	In one line
0. Prove the risky parts	Done	The Muse hook can block; Meta's API, Baseten and Modal sandboxes all work.
1. The guard	Done	Stops installs, including in scripts, bun/pipx and MCP servers; blocks whenever unsure.
2. The office and notebook	Done	Runs on Modal with the Supabase cache; the guard uses it by default.
3. Quick checks	Done	Real decisions in about a second: made-up and malicious packages blocked, established ones allowed.
4. Test lab	Next	Harmless fake-malware packages to test against.
5 to 8. Lab and detective	Later	Sandbox runs, version diffs, the Spark investigator, trigger-aware re-runs.
9 to 13	Later	Quick sorter, live report page, evaluation, live scan, ship and submit.

We are about two days behind the plan's schedule, but the riskiest unknowns are all settled. Submission target: Wednesday 4 November.

12. Words you'll see

Word	Meaning
Package	Ready-made code someone published for others to install (npm for JavaScript, PyPI for Python).
Hook	A program Muse runs before every action. Ours can say stop.
MCP server	An add-on tool for the AI helper. Adding one can also download and run a package.
Verdict	The office's answer: allow, block or ask (ask means the human decides).
Cache	The notebook of verdicts already given, so we don't judge the same version twice.
Fail closed	When something breaks, keep the door shut rather than open.
Sandbox	A sealed, throwaway computer where suspicious code can run without hurting anything.
Canary	Fake bait (like a fake password). If it leaves the sandbox, the package is guilty.
Slopsquatting	AI helpers sometimes invent package names that don't exist; attackers register those names with malware.
Typosquat	A malicious package named like a popular one with a small typo (axois instead of axios).
Install script	Code a package runs automatically the moment it's installed. Malware loves this.
OSV	A public database of known security problems, including a list of known malicious packages.
Provenance	A signature showing a package was built by a public CI system from its public code, not on someone's laptop.
Version range	A request like ^4, meaning "any 4.x version". The office works out exactly which version would be installed.
Open weights	An AI model whose recipe is public, so anyone can run it.
Cold start	The few seconds a sleeping cloud service takes to wake up on its first call.